

LocalRoots Water Watch

คู่มือ Fork ไปใช้กับพื้นที่ของคุณ

เปิด repo - Fork - แก้ config - ใส่จุดวัดน้ำ/กล้อง - ทดสอบ - Deploy

<https://github.com/tongfb/localroots-water-watch>

ต้องมีอะไรบ้าง GitHub account, Cloudflare account, Node.js และเครื่องที่เปิด Terminal/PowerShell ได้

ทำตามนี้ก่อน

- Fork repository ไปที่ GitHub account ของคุณ
- แก้ config/localroots.config.json ให้เป็นชื่อพื้นที่ของคุณ
- ใส่จุดวัดน้ำที่มี JSON API ที่ตรวจสอบแล้ว
- ใส่กล้องสาธารณะที่อนุญาตให้เปิดดู/นำภาพมาใช้ได้
- รัน npm install และ npm run verify
- เปิดดูด้วย npm run dev
- เมื่อข้อมูลถูกต้องแล้วจึง npm run deploy

หลักสำคัญ ทุกตัวเลขควรมี sourceUrl และเวลาของข้อมูลเมื่อแหล่งต้นทางมีให้ อย่าตั้ง threshold เตือนภัยเองถ้าหน่วยงานต้นทางไม่ได้ประกาศนิยามไว้

1. Fork repository

เข้า repo แล้วกด **Fork** จากหน้า GitHub จากนั้นเลือก account ของคุณให้ GitHub สร้างสำเนาโครงการใหม่

```
https://github.com/tongfb/localroots-water-watch
```

Clone repo ที่ fork มาไว้ในเครื่องของคุณ เช่น

```
git clone https://github.com/YOUR-GITHUB-NAME/localroots-water-watch.git
cd localroots-water-watch
```

อย่าแก้ไขของต้นฉบับ ทำงานใน repo ที่ fork ไปแล้ว เพื่อให้คุณดูและข้อมูลพื้นที่ของตัวเองและดึงอัปเดตจากต้นฉบับได้ง่าย

2. เปลี่ยนชื่อและข้อมูลพื้นที่

เปิดไฟล์ config/localroots.config.json แล้วเปลี่ยนค่าพื้นฐานให้เป็นพื้นที่ของคุณ

```
{
  "project": "my-area-water-watch",
  "title": "My Area Water Watch",
  "area": "อำเภอ/จังหวัดของคุณ",
  "timezone": "Asia/Bangkok",
  "refreshSeconds": 300,
  "theme": {
    "accent": "#0f766e",
    "subtitle": "ข้อมูลน้ำและสถานการณ์พื้นที่ของเรา"
  },
  "disclaimer": "โปรดยึดประกาศเตือนภัยและคำสั่งจากหน่วยงานทางการเป็นหลักเมื่อมีเหตุฉุกเฉิน"
}
```

แก้ไขเฉพาะค่าของพื้นที่ ไม่ต้องแก้ไข core ถ้ายังไม่จำเป็น

3. เพิ่มจุดวัดน้ำของพื้นที่

แต่ละจุดวัดน้ำเพิ่มใน array ชื่อ `jsonMetrics` โดย `core` จะอ่าน JSON จาก URL แล้วหาค่าตาม `valuePath` และ `timePath`

```
"jsonMetrics": [
  {
    "id": "river-main",
    "group": "ระดับน้ำ",
    "label": "สถานีหน้าตลาด",
    "url": "https://example.go.th/api/station/ABC123",
    "valuePath": "data.0.level",
    "timePath": "data.0.observed_at",
    "unit": "m",
    "sourceUrl": "https://example.go.th/stations/ABC123",
    "note": "ระดับน้ำจากสถานี ABC123"
  }
]
```

แต่ละช่องหมายถึงอะไร

ช่อง	ใส่อะไร
id	ชื่อภายในระบบ ควรไม่ซ้ำ เช่น river-main, canal-east
group	กลุ่มที่จะแสดง เช่น ระดับน้ำ, ฝน, อ่างเก็บน้ำ
label	ชื่อที่คนในพื้นที่เข้าใจกัน
url	URL ของ JSON API ที่ Worker จะเรียก
valuePath	ตำแหน่งค่าที่ต้องการใน JSON เช่น data.0.level
timePath	ตำแหน่งเวลาที่ข้อมูลถูกวัด เช่น data.0.observed_at
unit	หน่วย เช่น m, cm, mm
sourceUrl	หน้าต้นทางสำหรับให้คนกดตรวจสอบ
note	คำอธิบาย datum/สถานี/บริบทที่จำเป็น

valuePath อ่านยังไง

```
{
  "data": [
    { "level": 4.25, "observed_at": "2026-09-25T13:00:00+07:00" }
  ]
}
```

ตัวอย่างนี้ใช้ `valuePath = data.0.level` และ `timePath = data.0.observed_at` โดยเลข 0 หมายถึงรายการแรกใน array `data`

สำคัญ `jsonMetrics` ต้องเป็น endpoint ที่คืน JSON จริง ถ้ามีแค่หน้าเว็บ HTML หรือแอปที่ดูด้วยตา ให้ใส่เป็น links ไปก่อน หรือทำ adapter เพิ่มภายหลัง

4. เพิ่มหลายสถานี

ถ้ามีหลายจุดวัดน้ำ ให้เพิ่ม object ต่อกันใน `jsonMetrics` และคั่นด้วย `comma`

```
"jsonMetrics": [  
  {  
    "id": "river-upstream",  
    "group": "ระดับน้ำ",  
    "label": "สถานีต้นน้ำ",  
    "url": "https://example.go.th/api/A01",  
    "valuePath": "data.level",  
    "timePath": "data.time",  
    "unit": "m",  
    "sourceUrl": "https://example.go.th/A01"  
  },  
  {  
    "id": "river-town",  
    "group": "ระดับน้ำ",  
    "label": "สถานีในเมือง",  
    "url": "https://example.go.th/api/A02",  
    "valuePath": "data.level",  
    "timePath": "data.time",  
    "unit": "m",  
    "sourceUrl": "https://example.go.th/A02"  
  }  
]
```

เลือกสถานีให้มีความหมาย เลือกจุดที่ช่วยอธิบายการไหลของน้ำเข้าพื้นที่ จุดกลางพื้นที่ และจุดปลายน้ำ/ทางระบาย ถ้าพื้นที่ของคุณมีโครงสร้างแบบนั้น

ถ้า API ต้องใช้ key

```
{  
  "apiKeyEnv": "LOCAL_WATER_API_KEY",  
  "apiKeyQueryParam": "api_key"  
}
```

ถ้า API ต้องส่ง key ทาง header ให้ใช้ `apiKeyHeader` แทน `apiKeyQueryParam` และเก็บค่าจริงเป็น Cloudflare secret

5. เพิ่มกล้องของพื้นที่

Public Core รุ่นนี้รองรับภาพ snapshot และลิงก์ไปหน้าต้นทาง เพิ่มใน array `cameras`

```
"cameras": [  
  {  
    "label": "กล้องสะพานกลางเมือง",  
    "imageUrl": "https://example.go.th/camera/bridge.jpg",  
    "pageUrl": "https://example.go.th/camera/bridge",  
    "alt": "ภาพกล้องสะพานกลางเมือง",  
    "note": "ใช้ดูระดับน้ำและสภาพการจราจรบริเวณสะพาน"  
  }  
]
```

ถ้ามีหน้าดูกล้อง แต่ไม่มี URL ภาพ snapshot ที่ใช้ได้โดยตรง ให้ตัด `imageUrl` ออกและคง `pageUrl` ไว้ ระบบยังแสดงการ์ดพร้อมลิงก์ไปต้นทางได้

สิทธิ์การใช้งาน ใช้เฉพาะกล้องหรือภาพที่หน่วยงานอนุญาตให้สาธารณะเปิดดู/นำไปแสดงต่อได้ หากไม่แน่ใจให้ลิงก์ไปหน้า
ต้นทางแทนการดึงภาพมาแสดงเอง

6. เพิ่มลิงก์สำคัญของพื้นที่

แหล่งข้อมูลที่ไม่มี JSON API หรือเป็นหน้าอ้างอิงสำคัญ ให้ใส่ใน links

```
"links": [  
  {  
    "label": "ศูนย์เตือนภัยจังหวัด",  
    "url": "https://example.go.th/warning",  
    "note": "ประกาศเตือนภัยทางการ"  
  },  
  {  
    "label": "เรดาร์ฝนพื้นที่",  
    "url": "https://example.go.th/radar",  
    "note": "ใช้ประกอบการติดตามฝน"  
  }  
]
```

เช็กรอบแรกก่อนทดสอบ

- ชื่อพื้นที่และ subtitle ถูกต้อง
- ทุก jsonMetrics เปิด URL ได้และคืน JSON
- valuePath อ่านค่าที่ต้องการจริง
- timePath เป็นเวลาที่ parse ได้ ถ้าแหล่งข้อมูลมีเวลา
- sourceUrl กดแล้วพาคนไปดูต้นทางจริง
- กล้องทุกตัวเปิดได้ และไม่ละเมิดเงื่อนไขของต้นทาง
- ไม่มี API key, token หรือ secret ถูกเขียนลง config/repo

7. ติดตั้งและทดสอบ

```
npm install  
npm run verify  
npm run dev
```

เมื่อ npm run dev ทำงาน ให้เปิด URL local ที่ Wrangler แสดง แล้วตรวจว่าชื่อพื้นที่ จุดวัดน้ำ freshness source link และกล้องแสดงถูกต้อง

ถ้า verify ไม่ผ่าน แก้ error ให้หมดก่อน deploy โดยเฉพาะ JSON syntax เช่น comma เกิน/ขาด หรือ quote ไม่ครบ

8. Deploy ไป Cloudflare

```
npx wrangler login  
npm run deploy
```

ครั้งแรก Wrangler จะให้ยืนยัน Cloudflare account ผ่าน browser จากนั้น npm run deploy จะใช้คำสั่ง wrangler deploy ของโปรเจกต์

อย่าคัดลอกค่าของคนอื่น อย่าเอา Account ID, API token, domain, Worker name หรือ secret ของผู้ดูแลคนอื่นมาใช้กับ fork ของคุณ

ก่อนแจก URL ให้คนในพื้นที่

- ☐ หน้าเว็บระบุชื่อพื้นที่ถูก
- ☐ จุดวัดน้ำมาจากแหล่งต้นทางที่ตรวจสอบได้
- ☐ ค่าตัวเลขตรงกับ source
- ☐ เวลาข้อมูล/freshness แสดงสมเหตุผล
- ☐ ไม่ได้สร้างเกณฑ์เตือนภัยขึ้นเองโดยไม่มีนิยามจากหน่วยงาน
- ☐ กล้องเปิดได้และมีสิทธิ์ให้สาธารณะดู
- ☐ ไม่มี secret อยู่ใน GitHub
- ☐ npm run verify ผ่าน
- ☐ เปิด Worker URL หลัง deploy แล้วทดสอบอีกครั้ง

ถ้าแหล่งข้อมูลใหม่ไม่ตรงกับ core อย่าฟันแก้หลายจุดในระบบ ให้ทำ adapter แยกสำหรับแหล่งข้อมูลนั้น หรือเปิด issue/PR กลับมาที่ LocalRoots Water Watch เพื่อให้ pattern ใหม่สามารถนำกลับไปใช้กับพื้นที่อื่นได้

LocalRoots Water Watch - MIT License

Support the original project via Lightning: donate@zapm.uk

<https://github.com/tongfb/localroots-water-watch>